

Project 2 LLM-Powered Intelligent Agents

This project challenges students to design and implement an AI agent that uses a Large Language Model (LLM) (such as ChatGPT, Gemini, or LLaMA) to perform intelligent tasks that require understanding, reasoning, and communication in natural language.

Each agent should combine external data sources (e.g., web pages, documents, or datasets) with LLM reasoning to deliver meaningful outputs such as summaries, reports, analyses, or plans.

Basic Requirements: LLM-Powered Question Answering Agent

1. Web Interface (Front End)

- The system must provide a **simple web interface** (e.g., Flask, Django, or Streamlit app) where:
 - The user can **type a question** into an input box.
 - The user can:
 - Upload one or more files (PDF, TXT, DOC/DOCX, etc.), **or**
 - Provide one or more **website URLs** as information sources.
 - The user can click a **“Ask”** / **“Submit”** button to send the question.

2. Input Sources (Files + Websites)

- The system must accept the following file formats from the user:
 - **.pdf, .txt, .doc / .docx**
- The system must be able to use
 - Text content fetched from user-provided URLs (web pages) as the knowledge source for answering questions.

3. Processing and Answer Generation

- After the user submits a question:
 1. The system must extract text from the provided sources (uploaded files and/or web pages).
 2. The system must search or select relevant text from those sources (keyword search, simple chunking, or embeddings is fine at basic level).
 3. The system must call an LLM API (e.g., ChatGPT / OpenAI, Gemini / Google) with:
 - The user's question, and

- The relevant extracted text as context in the prompt (i.e., the LLM should answer *based on the provided material*).

4. Output (Answer Display)

- The LLM-generated answer must be:
 - Displayed back on the same web page under or next to the question.
 - Clearly labeled as the “Answer”.
- Show the source snippets or filenames/URLs that were used to generate the answer.
- Basic conversational memory for follow-up questions.

Intermediate Requirements

Build an end-to-end system with structured output and richer analysis.

Requirements:

1. Duplicate and store articles in SQLite or CSV file
2. Batch processing: summarize 10+ articles automatically.
3. Implement text chunking and embeddings using a vector store (FAISS, Chroma, or LangChain)
 - LLMs have context limits. You can't just dump 300 pages of lecture notes into the prompt every time the student asks a question.
4. When the user asks a question:
 - Retrieve the top-k relevant chunks of text by semantic similarity.
 - Construct a context-aware prompt that includes only retrieved snippets
5. Provide a simple chat-style interface:
 - Display the user question and model answer in conversation format.
 - Allow follow-up questions using recent context (basic short-term memory).
6. Visualization: simple chart (e.g., bar plot of sentiments by source).

Advanced Requirements

The agent can also read handwritten or scanned documents, interpret their contents, and answer user questions based on the extracted text.

Requirements

1. Multi-modal Input Support

- Accept scanned images or handwritten documents (JPG, PNG, or PDF).
- Apply OCR (e.g., Tesseract, Google Vision API, or EasyOCR) to extract raw text.
- Handle noisy or incomplete OCR output.

2. LLM-Based Information Extraction

- Use the LLM to identify and clean up important text elements such as *author names, dates, titles, institutions, keywords, or answers to specific questions*.
- Combine extracted information with user queries to form precise answers.

3. Integrated QA & Data Output

- The agent answers user questions *based on the recognized handwritten content* or other uploaded materials.
- The extracted structured information is exported to CSV, Excel, or DOCX files for record-keeping.

Submission Guidelines:

Please submit a single **.zip file** that includes the following:

1. ReadMe File

- Provide step-by-step instructions to reproduce your project results.
- Ensure that anyone can follow the instructions and run your source code successfully.
- Include details about system setup and environment configuration.

2. Project Report

The report should include:

- **Project description**
- **Functions implemented**
- **Technical details:** database server, software, programming languages, and scripts used
- **Highlighted features** of your project

- **Discussion:** reflect on which techniques/languages/scripts are more effective for specific tasks, and compare different approaches where applicable

3. **Source Code**

- All source code files required to run your project.